UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lecture 12: Generative AI II

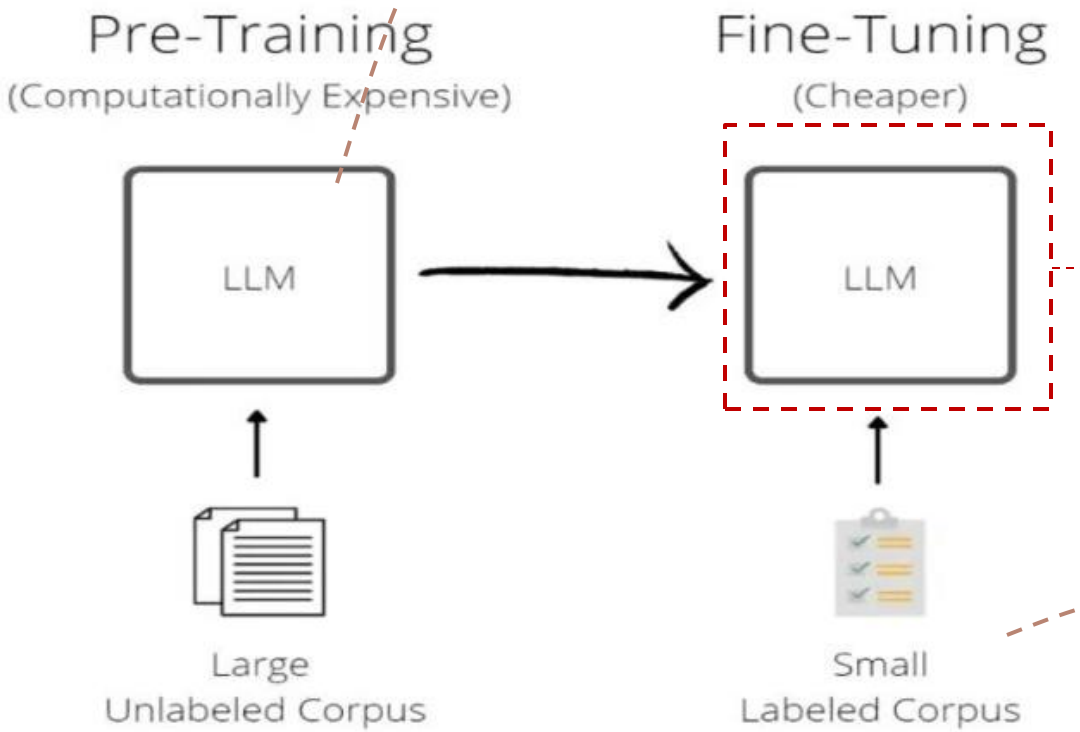
NLP: GPT; CV: VAE, GAN, and Diffusion Models

Instructor: Qian Liu

Assignment-3

- Pretraining Fine-tuning Paradigm

BERT-base-uncased version is used



Fine-tuned Model! It is used on downstream task (poem sentiment analysis)

Domain-specific Data: Poem training set

Assignment-3

Task-1: Load Data - 2 marks

Task 1.1: Load data - 1 mark

Note: please use the **official data split**.

Task 1.2: Display the statistical information about the dataset - 1 mark

Note: please provide the statistical information of data, e.g., average length of the input instances; distribution of labels in training data.

- To better understand your data!
  - Why "avg length of input instances"? --> Input limitation!
    - BERT-base and BERT-large models are typically limited to **512 tokens** per input sequence.
    - Too long (e.g., input is long documents) --> split into multiple pieces!
    - Too short (e.g., twitter, headlines) --> lower max_seq_length; If max_seq_length is **too large**, memory and computation are wasted on padding, especially in batch processing.
  - We should compute the length of words, tokens, or characters?
    - The standard answer is token!
    - Considering you may be unfamiliar to token, counting words is acceptable.
    - But character is **NOT** acceptable!! --> Characters are too small to capture meanings of input sequence.
    - Note that different pre-processing methods will lead to different average length-->Exact numbers aren't important — approximate ranges are what matter.
  - Why "distributions of labels of training data"?
    - To understand whether there is a data imbalance issue, which would affect model design

Task 2.1 BERT tokenizer!

UNIVERSITY OF AUCKLAND Waipapa Taumata Rau NEW ZEALAND | SCIENCE

Assignment-3

Task 2: Implementation - 5 marks

Task 2.1: BERT tokenizer - 1 mark

Please pick one training example and show what the BERT tokenizer outputs.

This task will help you get a better understanding of how the BERT tokenizer works.

Task 2.2: Model Training and Parameters Selection- 4 mark

Train your model. Discuss your selected hyperparameters and their impact on validation performance. The tuned parameters include:

- learning_rate,
- batch_size,
- num_train_epochs,
- warmup_steps

Note-1: **Please try at least two different parameter configurations**, and report their performance on the validation set. Here are two example:

Model-A :

- learning_rate: 3e-5
- batch_size: 16
- num_train_epochs: 3
- warmup_steps: 500

Validation Accuracy: xxxx

Model-B:

- learning_rate: 2e-5
- batch_size: 16
- num_train_epochs: 3
- warmup_steps: 0

Validation Accuracy: xxxx

🔧 **Example Parameters for Grid Search**

Let's say you're tuning:

- **Learning rate:** `[1e-5, 3e-5, 5e-5]`
- **Epochs:** `[3, 4, 5]`
- **Warm-up rate:** `[0.0, 0.1, 0.2]`

This creates a grid with:

3 × 3 × 3 = 27 combinations of hyperparameters.

- Tokenizer!
  - Word: unhappiness → tokenizer → tokens: "un", "##happy", "##ness"
- Model Training:
  - Option-1: Using Trainer() in huggingface
  - Option-2: handwritten Trainer()
- Parameter Selection!
  - How to make sure which setting is best?
  - We train models on different settings; then compare performance on validation set;
  - The parameter setting with the best performance on validation set is selected!
  - The performance on validation set is just a reference! It is not equal to the performance on test set.
  - To make the assignment easier, we only require you to try two different settings. In practice, you need to try more settings to have a good understanding of your model!
  - Popular solution: grid search!

Assignment-3

Task 3: Model Performance - 2 mark

Please report the used parameters, and the model performance on the test set:

- accuracy **>= 77%** will achieve 2 mark;
- accuracy **>= 75%** and accuracy **< 77%** will achieve 1.5 mark;
- accuracy **>= 70%** and accuracy **< 75%** will achieve 1 mark;
- accuracy **<70%** will achieve 0 mark.

Note that missing parameters will lead to a deduction of 0.5 points.

It's important to indicate which parameter configuration was applied during testing.

Task 4: Error Case Analysis - 1 mark

Please analysis the error cases of your trained model (200 words max).

1. We're very open with this analysis — the requirements aren't strict.
2. This step is just a reminder: when you're designing your AI model, it's important to do error analysis and **try to guess why the model might be failing**.
3. This is a **crucial step** for improving your model in the future.

UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lecture 12: Generative AI II

NLP: GPT; CV: VAE, GAN, and Diffusion Models

Instructor: Qian Liu

Concepts

- **Artificial Intelligence** (AI): machines or systems that can perform tasks that normally require human intelligence
- **Machine Learning** (ML): Subfield of AI --that learn patterns from data to make predictions or decisions without being explicitly programmed
- **Deep Learning**: Subfield of ML -- utilizing artificial neural networks with multiple layers for complex learning from massive datasets.
- **Generative AI**: Models to create new content based on input, including language, code, images, video, and so on.

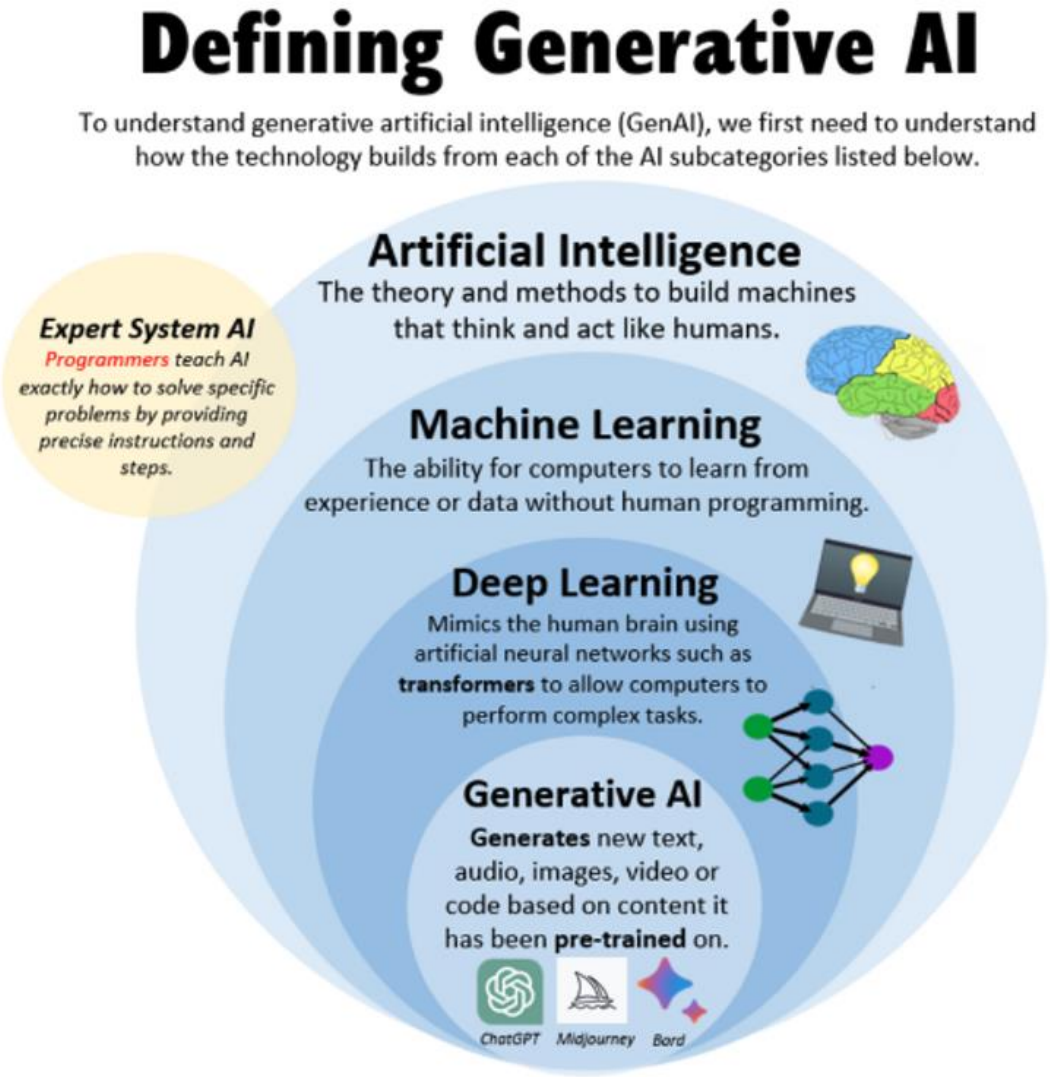
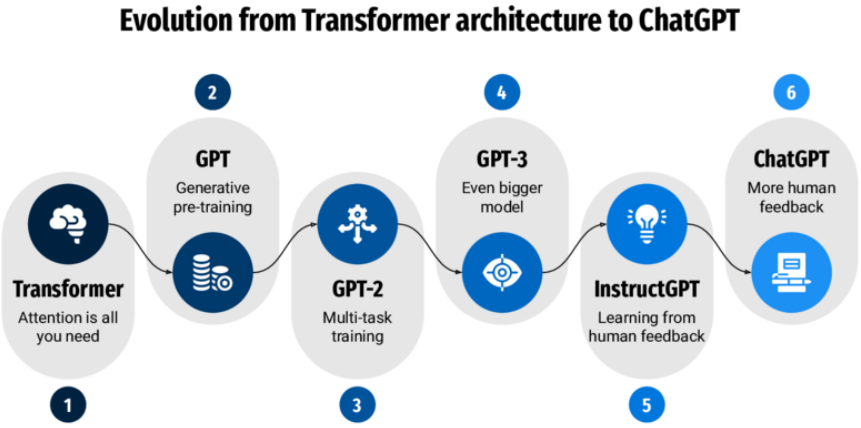
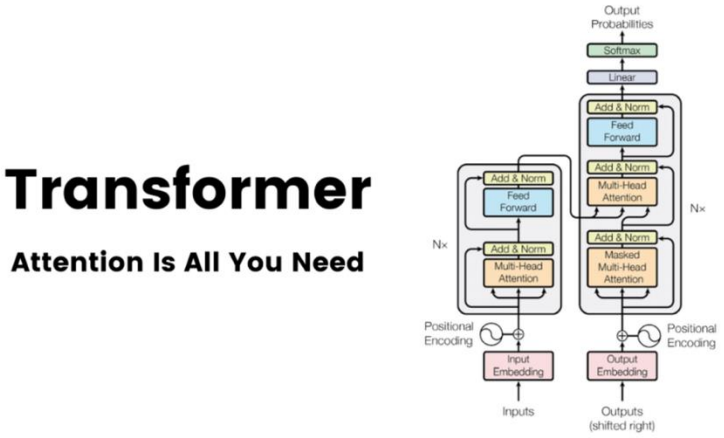


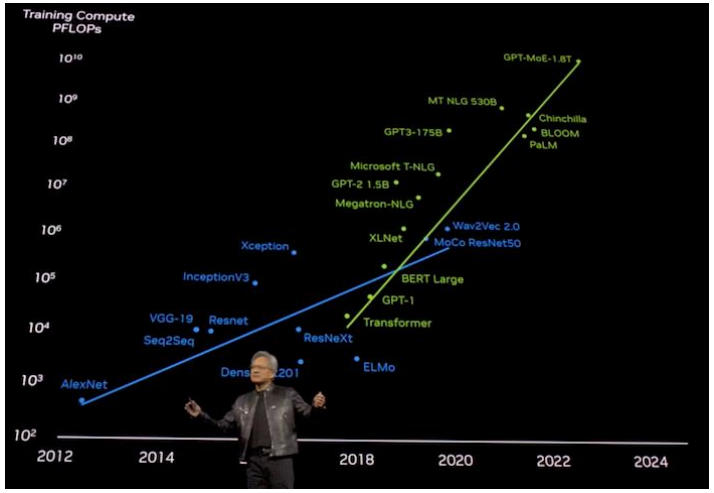
Fig from https://www.aiforeducation.io/ai-resources/generative-ai-explainer

Foundations of Generative AI

- Transformer Architecture
- Large-scale Pretraining + Self-Supervision
- Scalability: More Data + Bigger Models + Compute
- Instruction Tuning & RLHF







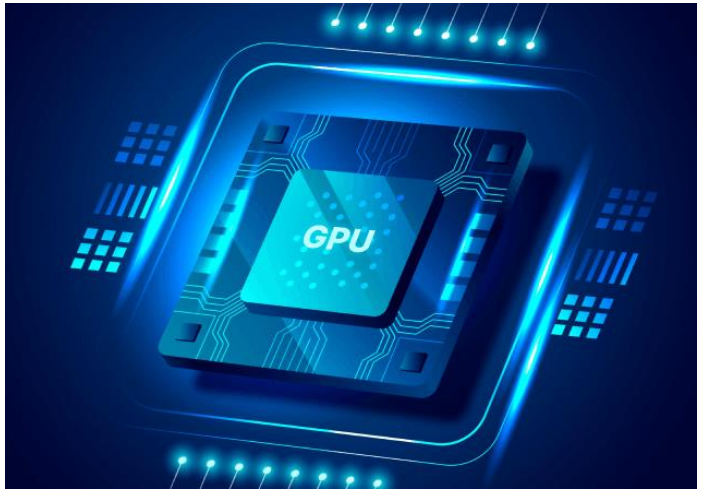


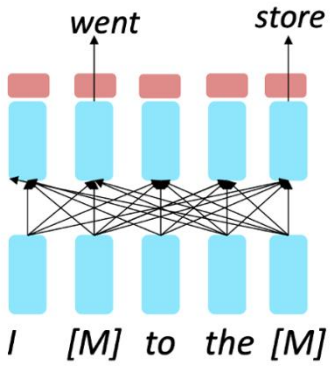
Image from https://medium.com/sage-ai/demystify-transformers-a-comprehensive-guide-to-scaling-laws-attention-mechanism-fine-tuning-fffb62fc2552

Text Generation

S = Where are we going

Previous words (Context) — Word being predicted

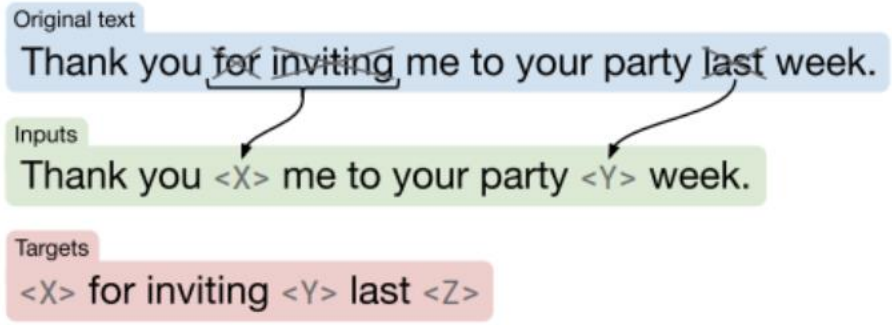
- Language Modeling!
  - Given an input sequence of words, language modeling is to predict the next word!
  - Other formats (designed for self-supervised learning):
    - BERT: masked language modeling
    - T5: span corruption
    - GPT: predicting the next word (token)



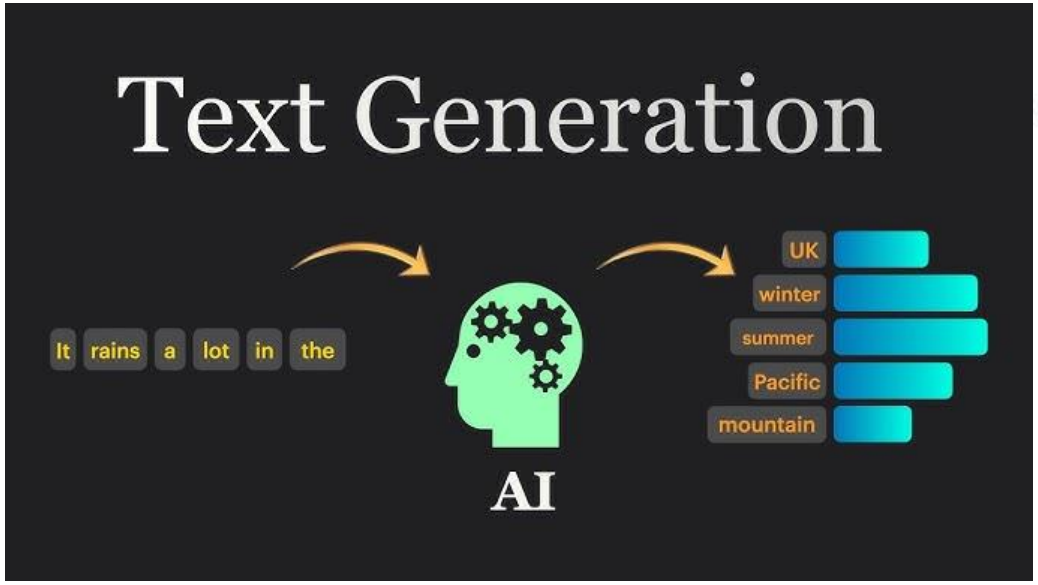
Text: Second Law of Robotics: A robot must obey the orders given it by human beings

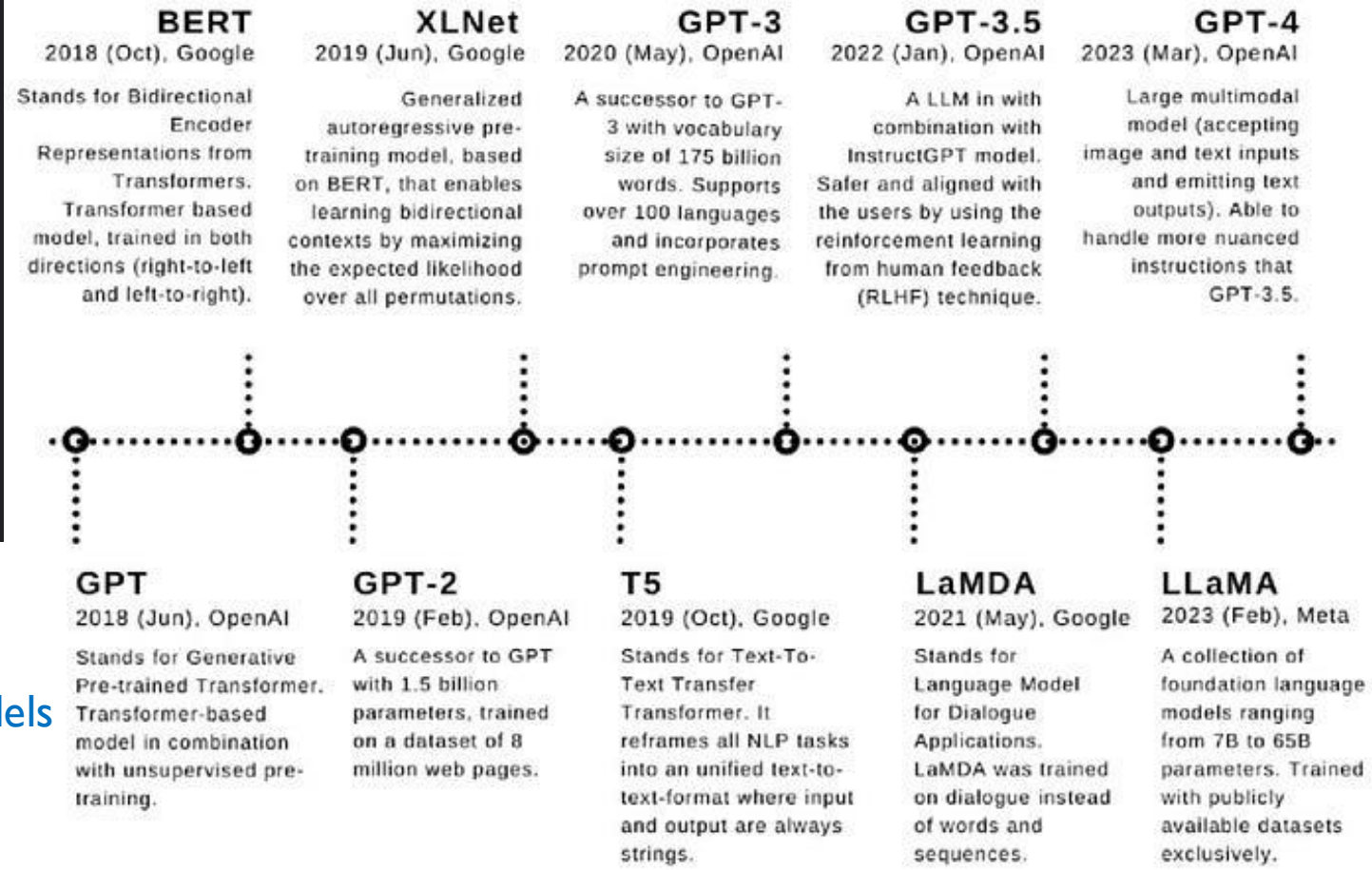
Generated training examples

Example #	Input (features)	Correct output (labels)
1	Second law of robotics :	a
2	Second law of robotics : a	robot
3	Second law of robotics : a robot	must
...		



Text Generation





Based on language models!

BERT/T5/GPT-1,2 …

- Commonly denoted as Pretrained Language Models
- Open-source

Large Language Models:

- Close-source: GPT-3.5, GPT-4
- Open-source: LLaMa…

Image from https://medium.com/@mukherjianu/a-brief-history-on-the-evolution-of-llms-and-the-future-of-ai-0c7153de28fc

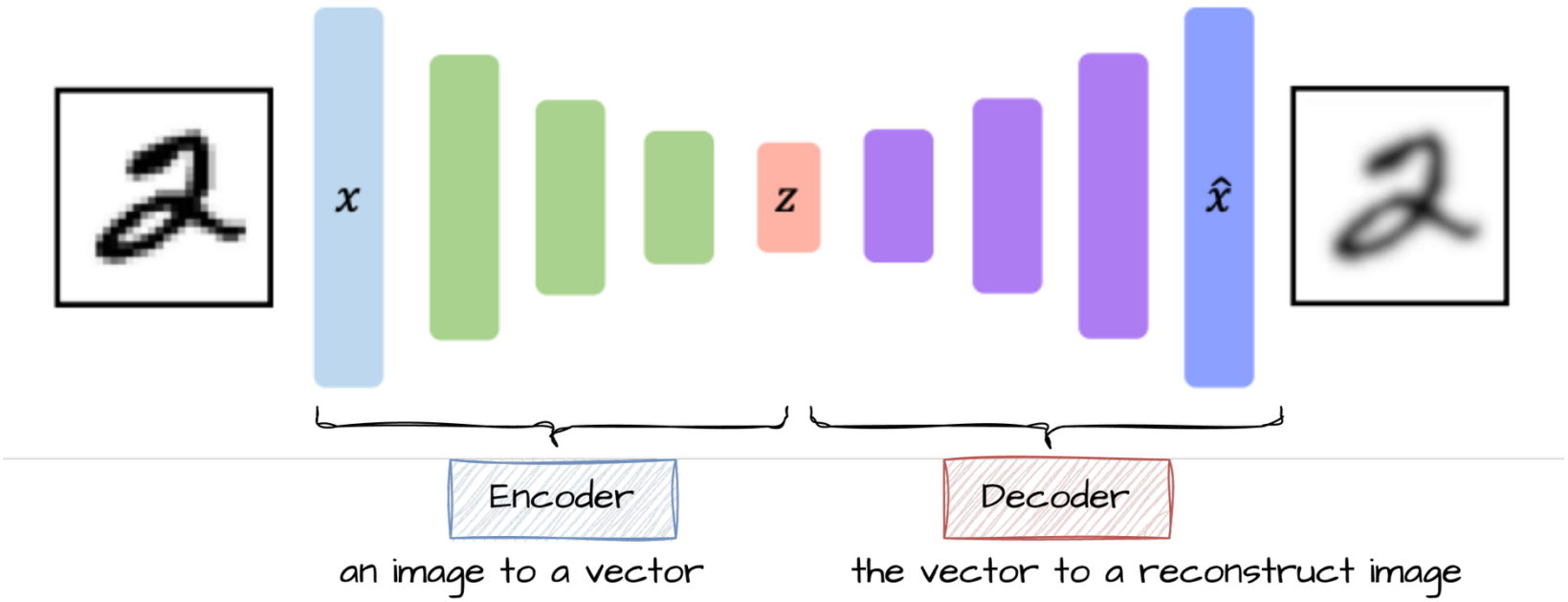
Image Generation

- Image generation is the task of creating realistic or meaningful images from input data using AI models.
- Representative Methods:
  - Autoencoder (AE)
  - Variational Autoencoder (VAE)
  - Generative Adversarial Network (GAN)
  - Diffusion Models

These pictures were generated by Stable Diffusion, a recent diffusion generative model.

Autoencoder

- **Autoencoder**: Unsupervised approach for learning a **lower-dimensional feature representation** from unlabeled training data
  - "**Encoder**" learns mapping from the data, x, to a low-dimensional latent space, z
  - **"Decoder"** learns mapping back from latent, z, to a reconstructed observation $\hat{x}$

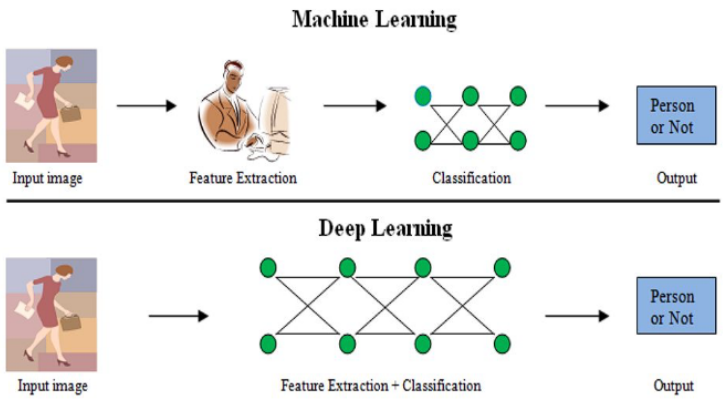


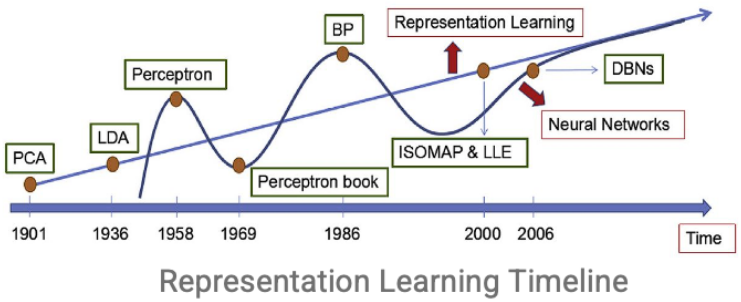
Training Objective:

Reconstruction -- The model tries to **minimize the difference** between the original input and its reconstruction

Recap: Feature

- **Feature**
  - AI models, especially machine learning models, **cannot** directly interpret raw data such as text, images, or audio in their original form.
  - Instead, they require input data to be converted into numerical representations—called features—that capture the relevant patterns or properties of the data.





Representation Learning Timeline

https://viso.ai/deep-learning/representation-learning/

Various names:

- Feature
- Representation
- Embedding
- Hidden vector
- …

- Feature space
- Latent space
- Embedding space
- Hidden space
- …

14

Autoencoder

- Autoencoder is a form of compression!
  - Reconstruction loss forces the latent representation to capture (or encode) as much "information" about the data as possible
  - Dimensionality Reduction: The encoder reduces the dimensionality of input data into a lower-dimensional latent space.
  - The compressed representation learned by autoencoders can be used for tasks requiring concise data representation.

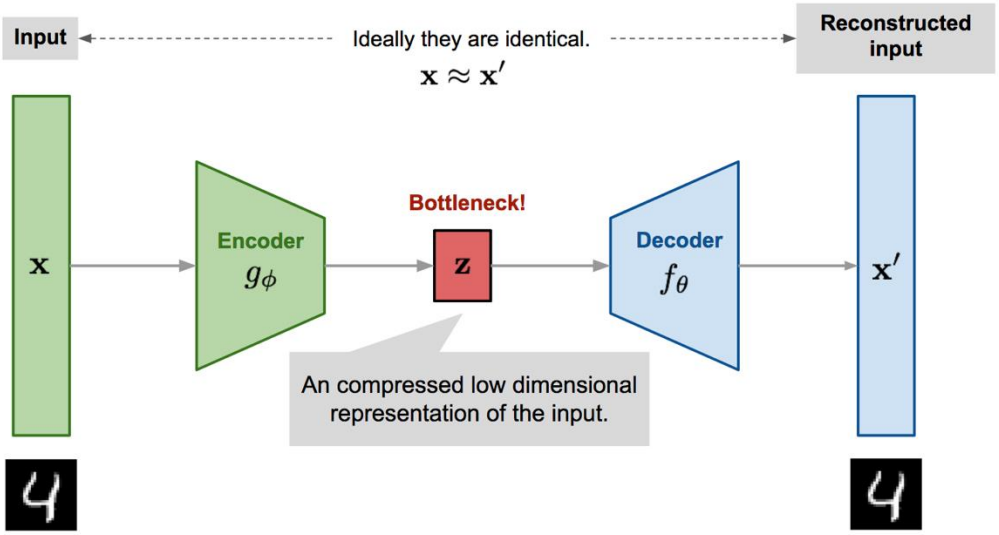
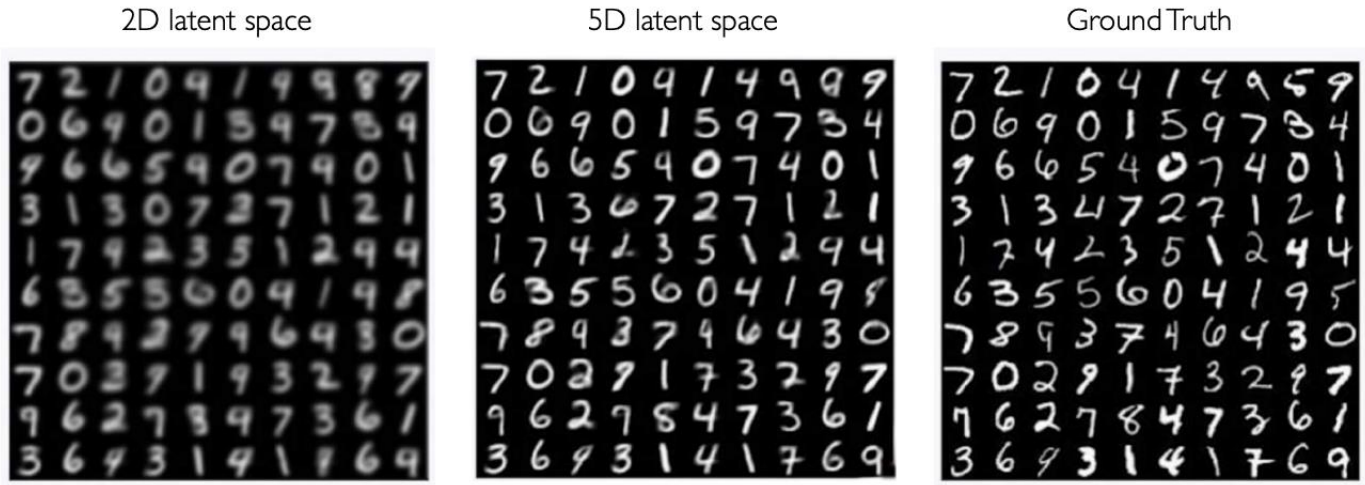


Image in space $x, \hat{x} \in R^m$
Encoder data in latent space $z \in R^n$
Dimensionality reduction: $m \gg n$

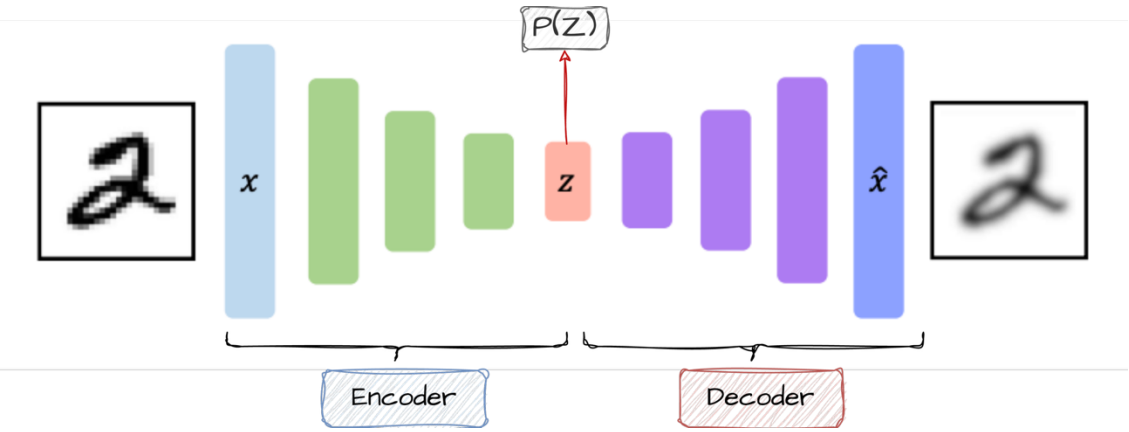


lossy encoding
Some information is lost when reducing the number of dimensions

lossless encoding
Almost no information is lost when reducing the number of dimensions

Variational Autoencoder (VAE)

- **Variational autoencoders (VAEs)** are a class of generative and probabilistic models that extend the concept of autoencoders (AEs) by introducing probabilistic encoding and decoding.
  - AEs map inputs to a fixed latent representation
  - VAEs model the latent space as a probability distribution



encoded representations z in the latent space are indeed assumed to follow the prior distribution p(z)

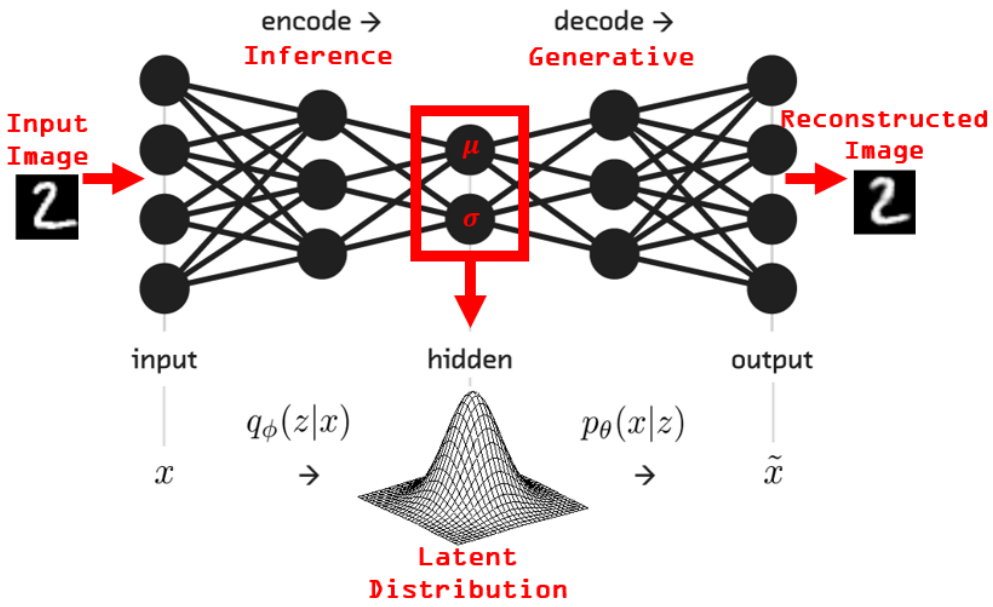
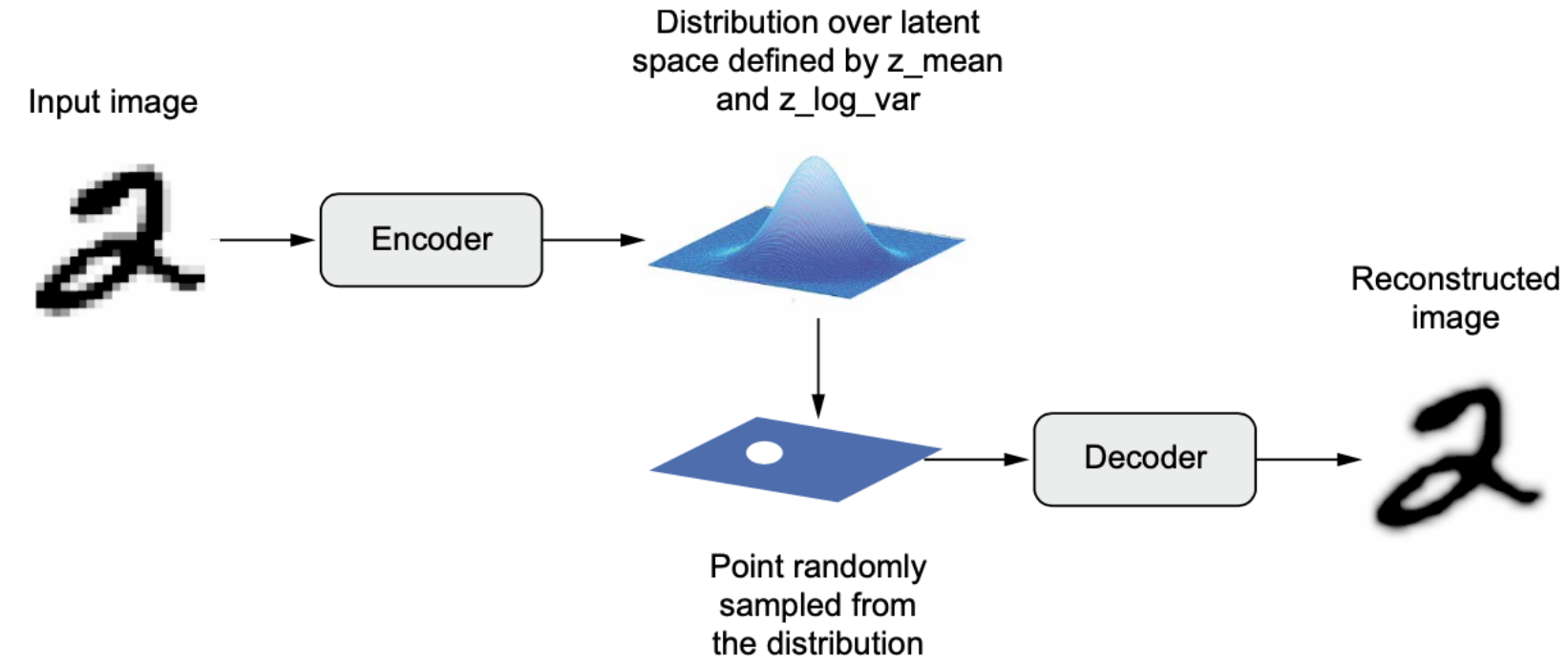


Fig from https://vitalflux.com/autoencoder-vs-variational-autoencoder-vae-difference/#What_are_Variational_Autoencoders_VAE

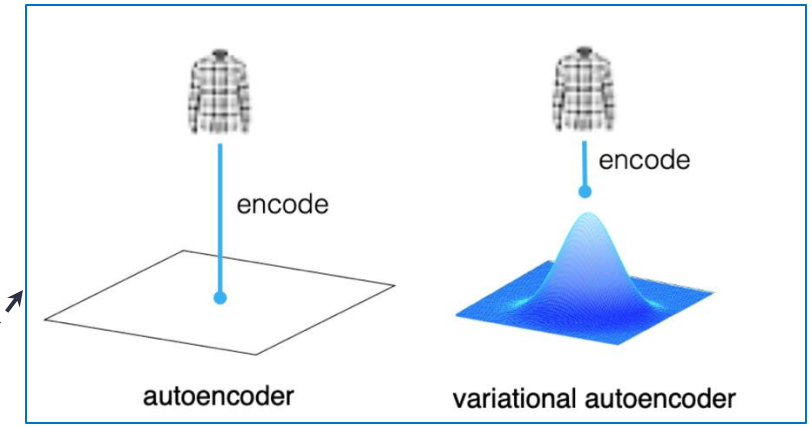
Variational Autoencoder (VAE)

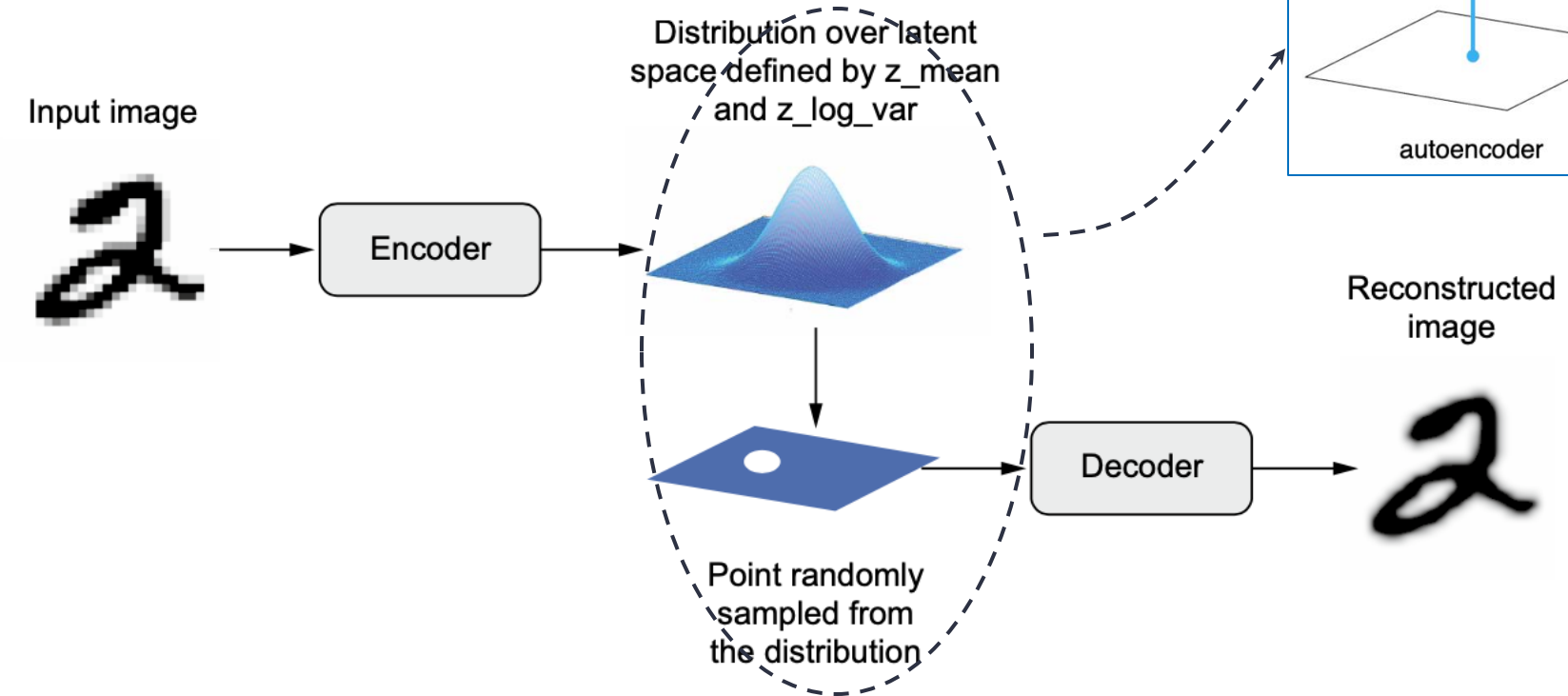
- **Variational autoencoders (VAEs)** are a class of generative and probabilistic models that extend the concept of autoencoders (AEs) by introducing probabilistic encoding and decoding.
  - AEs map inputs to a fixed latent representation
  - VAEs model the latent space as a probability distribution



Variational Autoencoder (VAE)

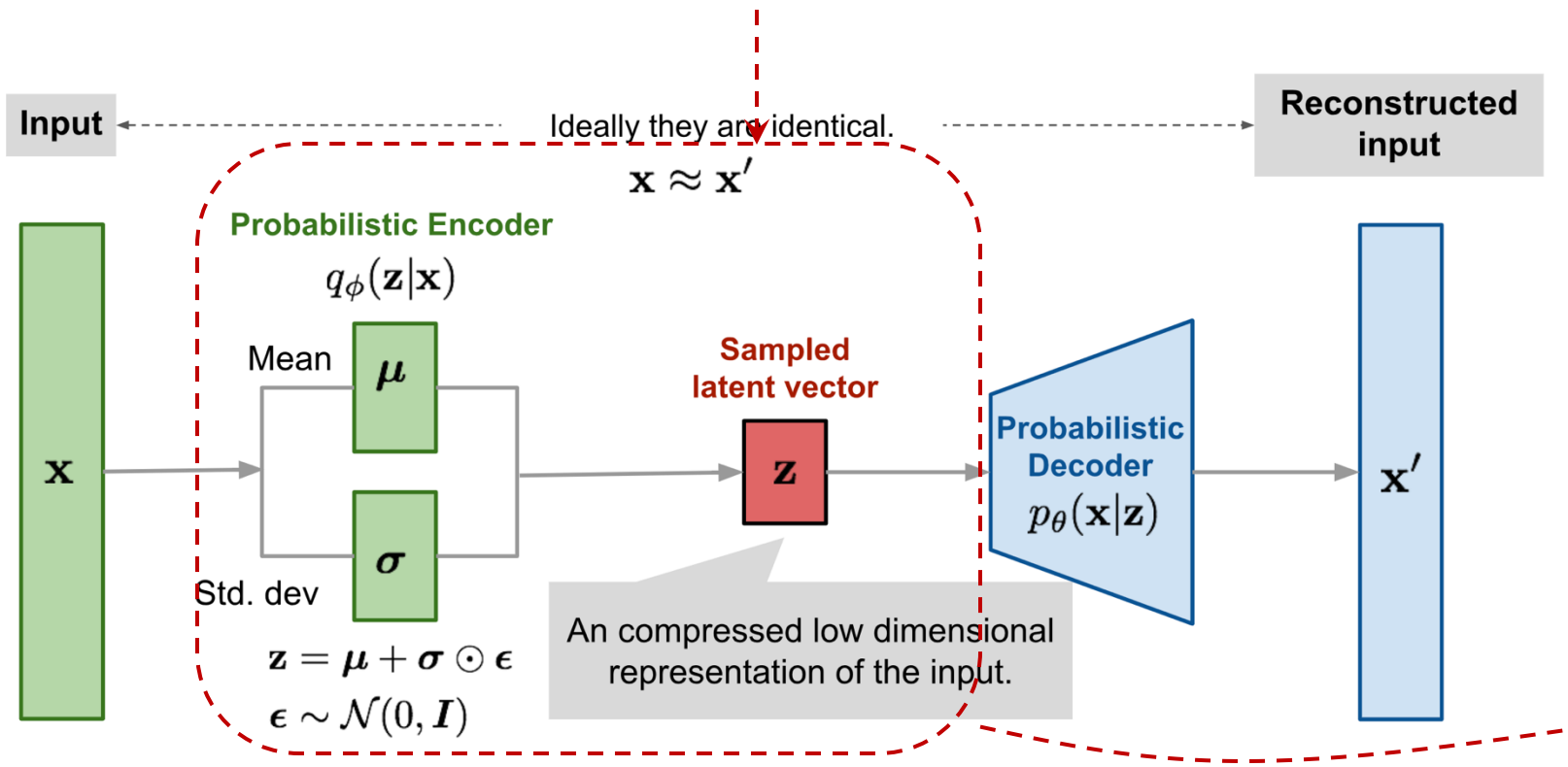
- **Variational autoencoders (VAEs)** are a class of generative and probabilistic models that extend the concept of autoencoders (AEs) by introducing probabilistic encoding and decoding.
  - AEs map inputs to a fixed latent representation
  - VAEs model the latent space as a probability distribution





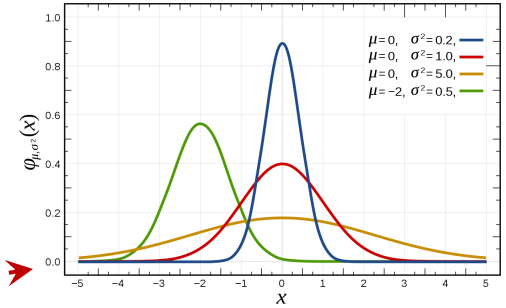
Variational Autoencoder (VAE)

- **Variational autoencoders (VAEs)** are a class of generative and probabilistic models that extend the concept of autoencoders (AEs) by introducing probabilistic encoding and decoding.
  - AEs map inputs to a fixed latent representation
  - VAEs model the latent space as a probability distribution

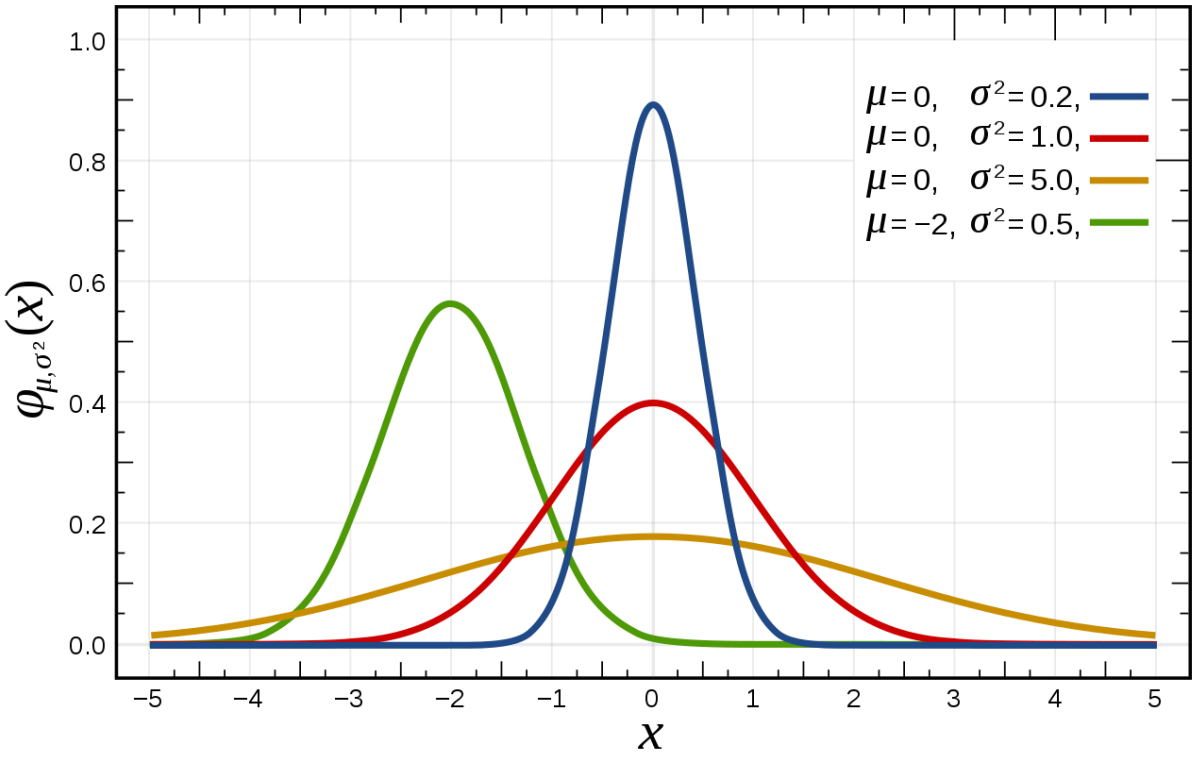


Training Objective:

1. **Reconstruction**
2. **KL divergence** (to keep latent distribution close to a prior, typically a standard Gaussian)



Gussian Distribution



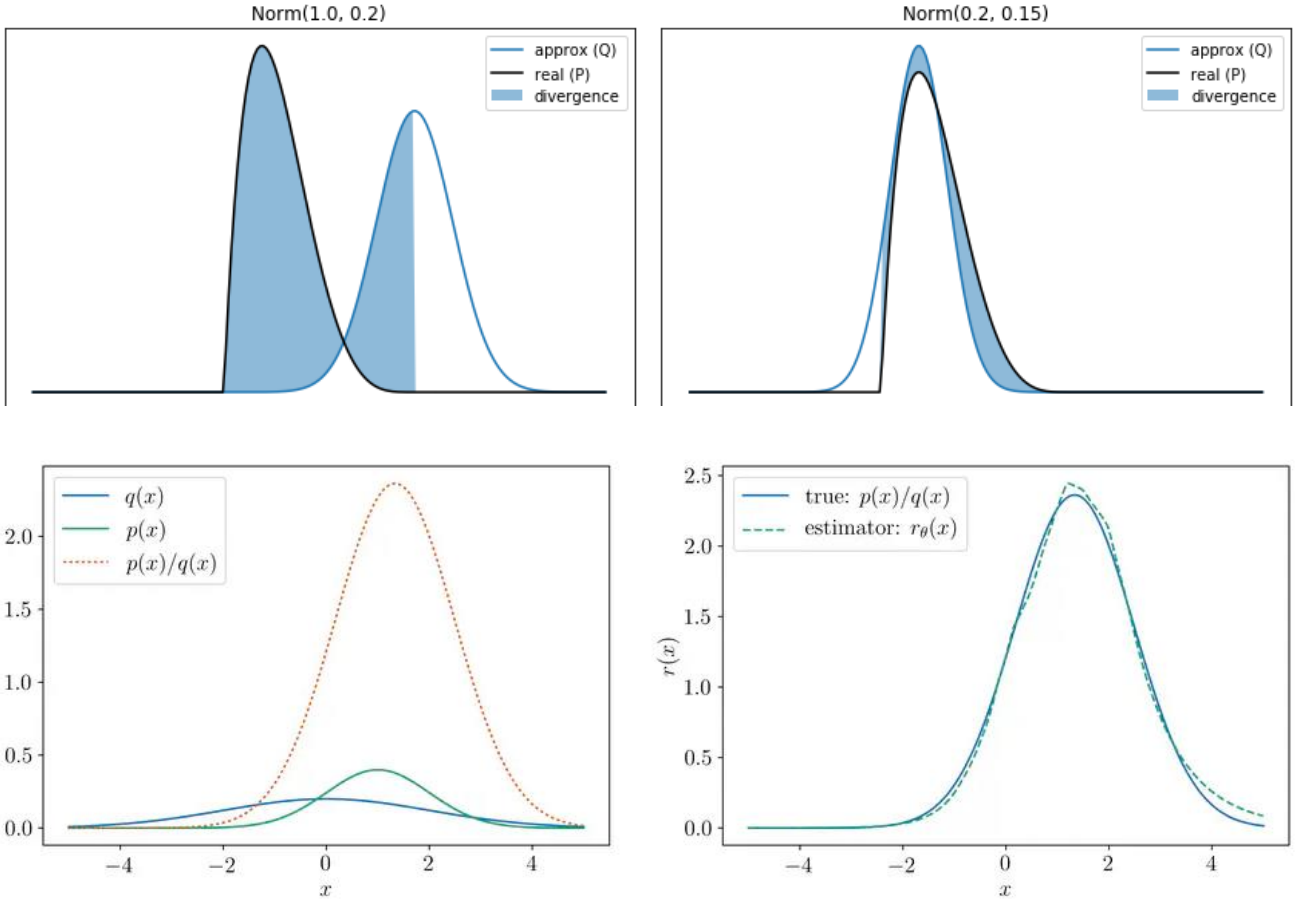
In probability theory and statistics, a **normal distribution** or **Gaussian distribution** is a type of continuous probability distribution for a real-valued random variable. The general form of its probability density function is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

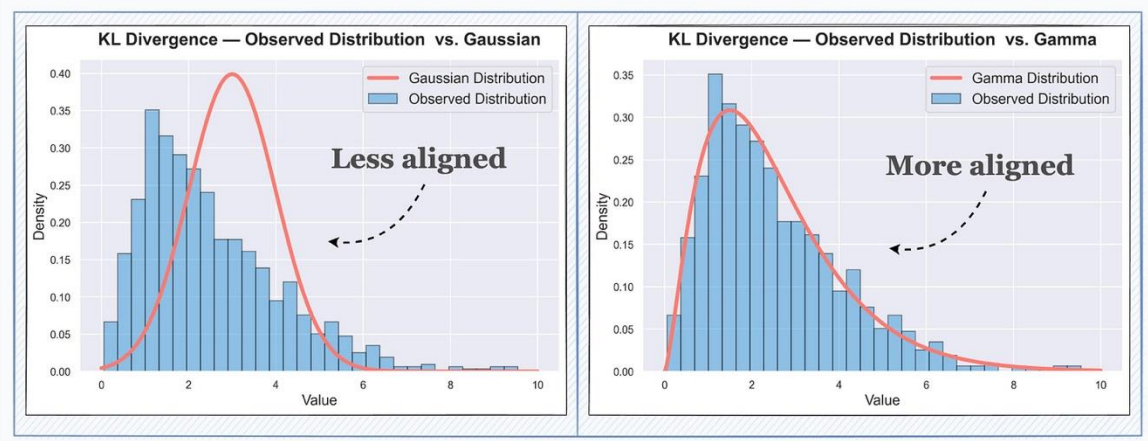
The parameter μ is the mean or expectation of the distribution (and also its median and mode), while the parameter σ is its standard deviation. The variance of the distribution is σ^2. A random variable with a Gaussian distribution is said to be **normally distributed**, and is called a **normal deviate**.

https://en.wikipedia.org/wiki/Normal_distribution

KL Divergence



KL divergence (short for **Kullback–Leibler divergence**) is a fundamental concept in information theory and statistics. It measures how **one probability distribution diverges from a second, expected probability distribution**.



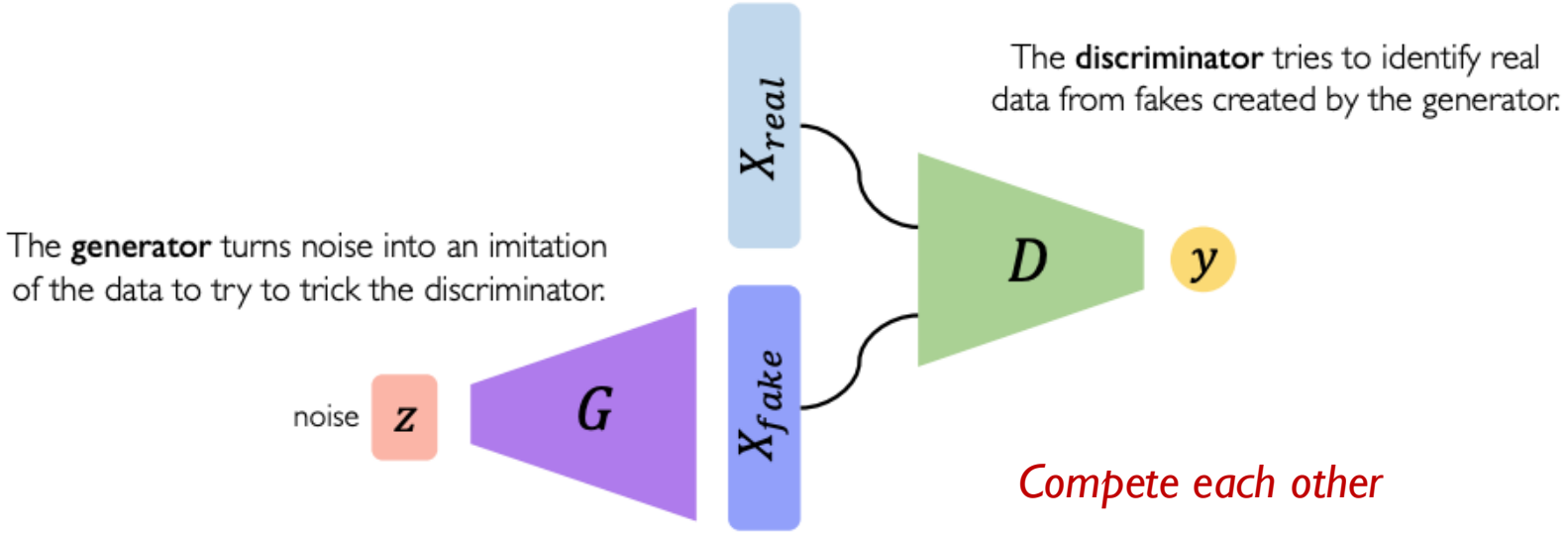
KL Divergence = 78.9 ❌

KL Divergence = 46.3 ✅

https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence

Generative Adversarial Network (GAN)

- GANs learn to generate new data similar to a training dataset.
- It consists of two networks, a generator and a discriminator, that plays a two-player game.
  - The generator takes in random values sampled from a normal distribution and produces a new sample,
  - The discriminator tries to distinguish between the real and generated sample.

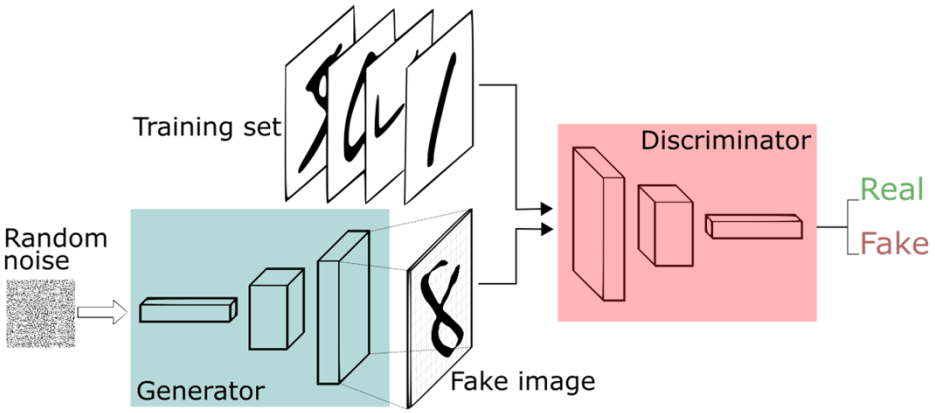


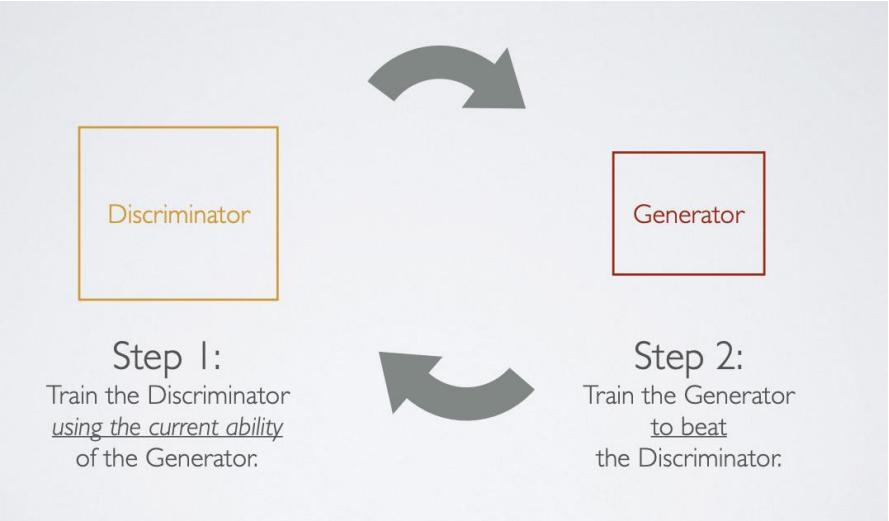
- Training by adversarial loss.
- Generator aims to "fool" a Discriminator by generating samples that are indistinguishable from real ones.
- The aim is to make the discriminator unable to differentiate between true and generated samples.

Paper of Generative Adversarial Networks https://arxiv.org/abs/1406.2661

Generative Adversarial Networks (GANs)

- How to train a GAN?
  - Which network should I train first?
    - Discriminator!
    - What is the training data?
      - The Discriminator is a Binary classifier.
      - The Discriminator has two class - Real and Fake.
      - The data for Real class if already given: THE TRAINING DATA
      - The data for Fake class? -> generate from the Generator
  - What's next?
    - Train the Generator!
    - How to train? What is the training objective?
      - Generate images from the Generator such that they are classified incorrectly by the Discriminator!



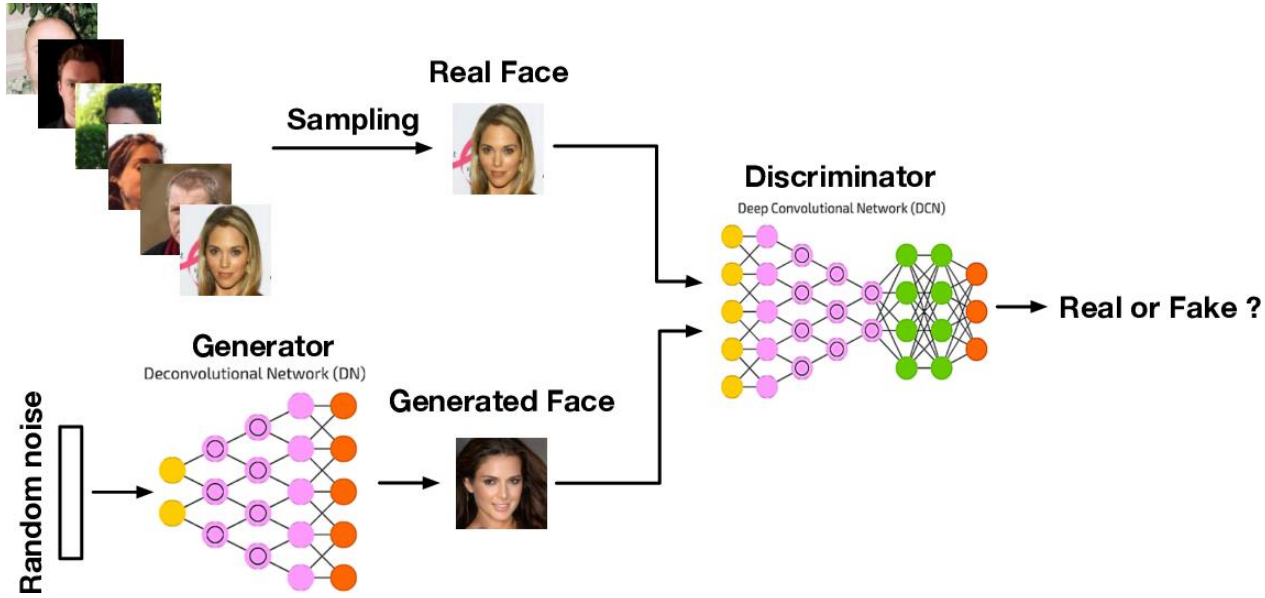


Generative Adversarial Networks (GANs)

- One of the main disadvantages of GANs is:
  - **Training instability** — GANs are difficult to train and often suffer from issues like mode collapse, where the generator produces limited varieties of outputs.

Because GANs involve two networks (generator and discriminator) in a **minimax game**, balancing them is tricky:

- If the discriminator becomes too strong, the generator can't learn.
- If the generator tricks the discriminator too easily, training stops improving.



Diffusion Models

- Diffusion models excel at generating realistic and coherent content based on textual prompts and efficiently handling image transformations and retrievals.
- Popular applications include Stable Diffusion, DALL-E 2, and Imagen.

Dall-E-2

https://cdn.openai.com/papers/dall-e-2.pdf

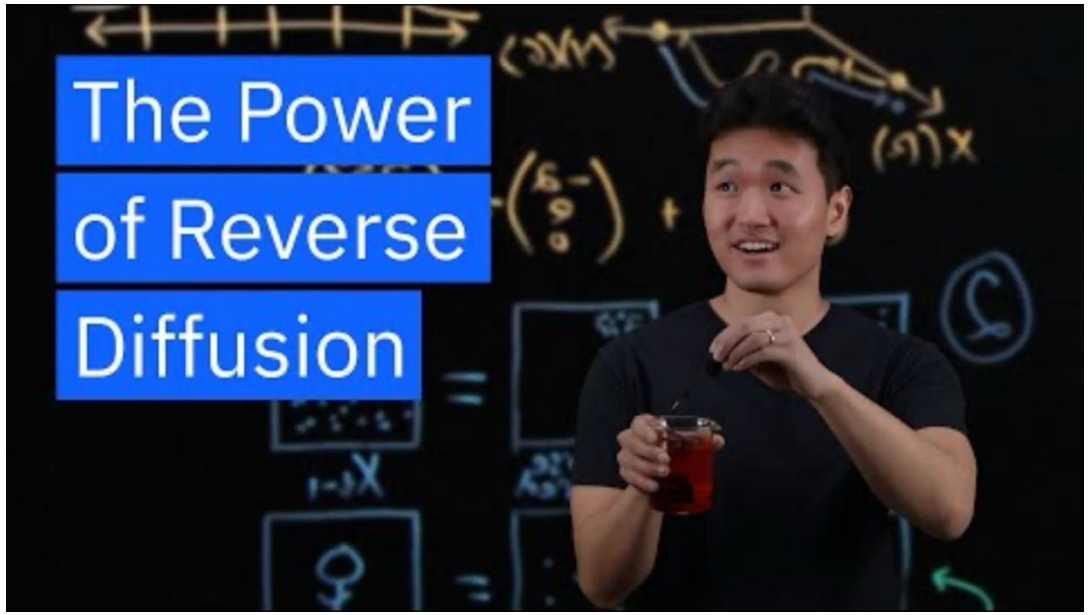
Imagen

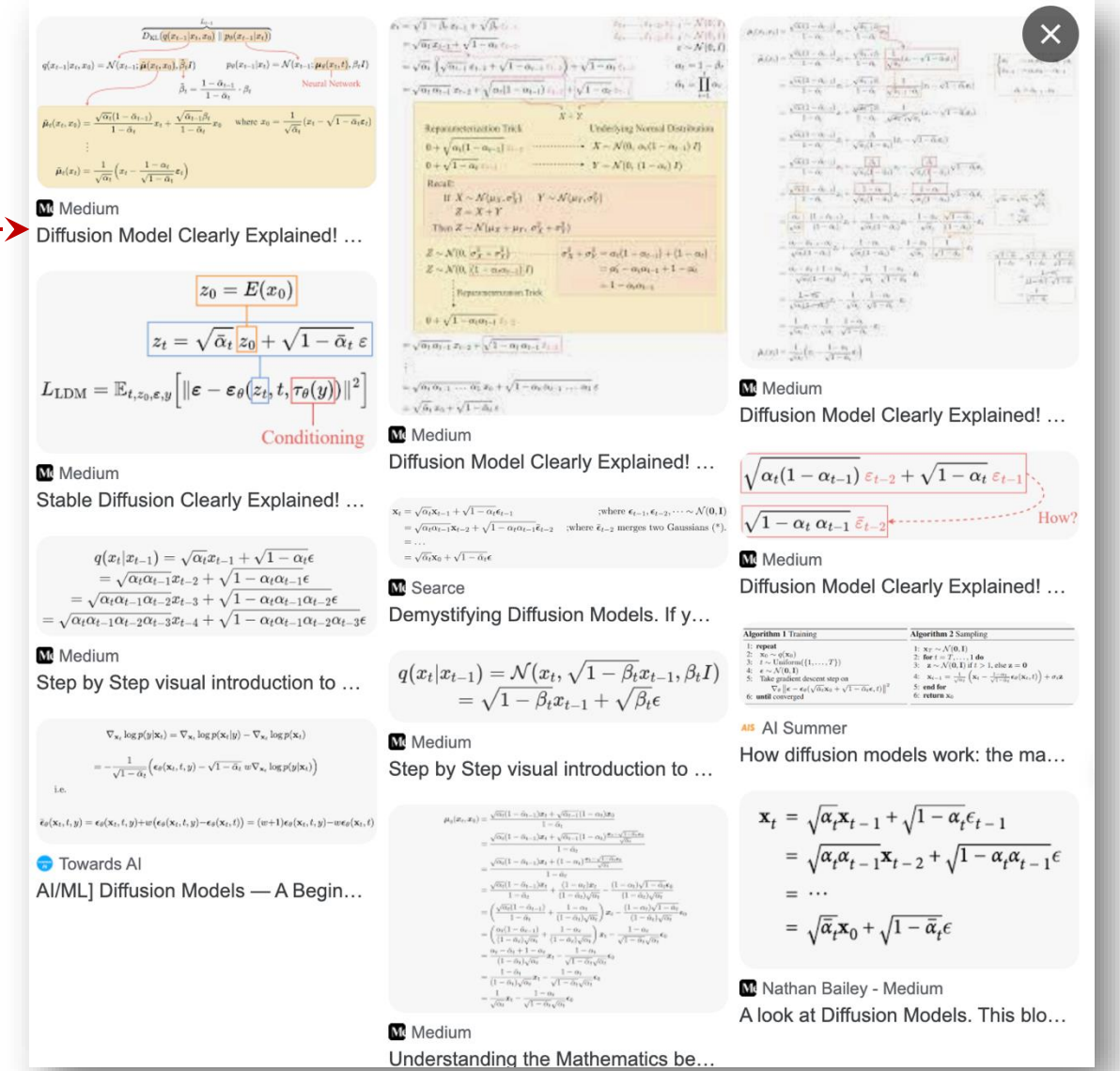
Diffusion Models

Stable diffusion is a popular approach for image generation that uses diffusion models (DMs) and the efficiency of latent space representation. The method introduces a two-stage training process to enable high-quality image synthesis while overcoming the computational challenges associated directly operating in pixel space.

Diffusion Models

- Lots of mathematics!
- Readings:
  - https://www.ibm.com/think/topics/diffusion-models
  - https://arxiv.org/abs/2209.00796

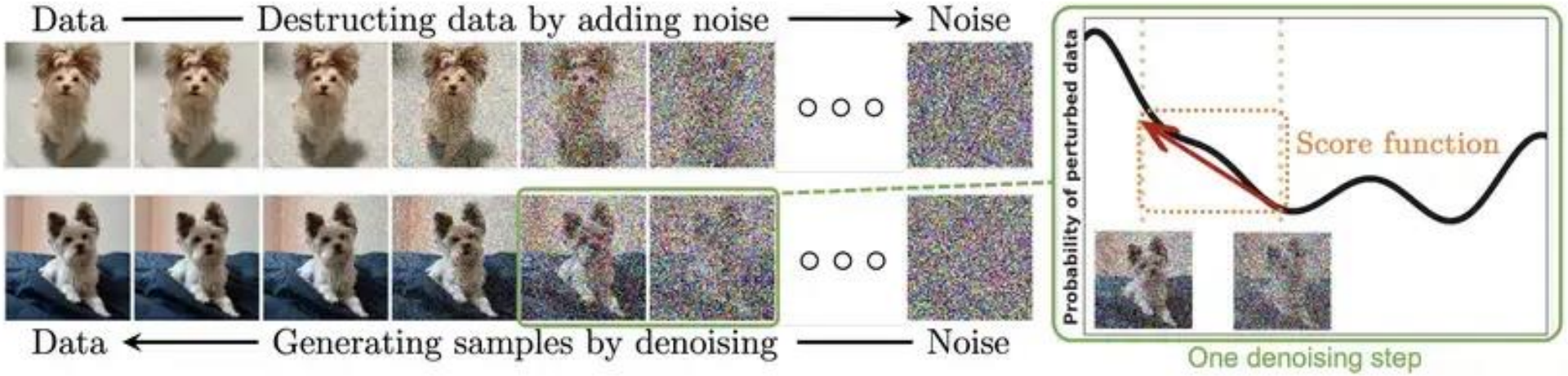




If you search it using Google…

Diffusion Models

- The process of training and deploying a diffusion can be broken down into three key stages:
  1. **The forward diffusion process,** wherein an image from the training data set is transformed into pure noise—usually a gaussian distribution.
  2. **The reverse diffusion process,** wherein the model learns the inverse of each previous step in the original forward diffusion process
  3. **Image generation,** wherein the trained model samples a random noise distribution and transforms it into a high-quality output by using the reverse diffusion process it has learned to denoise a random sample of gaussian noise.



Diffusion Models

1. The forward diffusion process, wherein an image from the training data set is transformed into pure noise—usually a gaussian distribution.

- Forward diffusion (noising)
  - $x_0 \rightarrow x_1 \rightarrow \cdots x_T$
  - Take a data distribution $x_0 \sim p(x)$, turn it into noise by diffusion $x_T \sim \mathcal{N}(0, \sigma^2 I)$

$\boldsymbol{x_0}$ $\boldsymbol{x_1}$ $\boldsymbol{x_{T-1}}$ $\boldsymbol{x_T}$

  - The forward diffusion process is a multi-step process that gradually adds a small amount of Gaussian noise to the sample until it becomes white noise.
    - A commonly used value for the number of steps is 1000 !

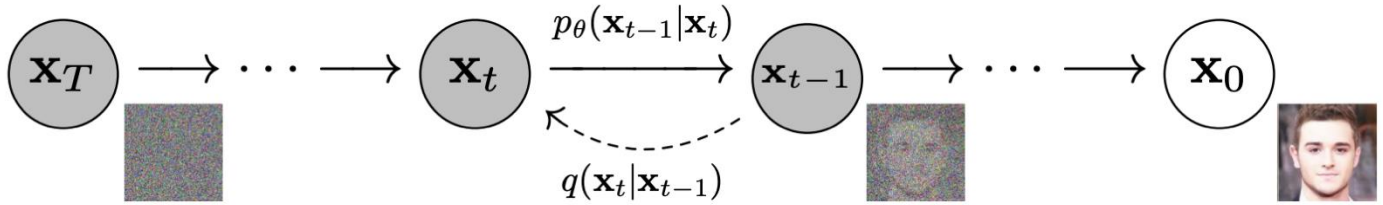
Diffusion Models

2. The reverse diffusion process, wherein the model learns the inverse of each previous step in the original forward diffusion process

- Revise diffusion (denoising)
  - $x_T \rightarrow x_{T-1} \rightarrow \cdots x_0$

$\boldsymbol{x_0}$ $\boldsymbol{x_1}$ $\boldsymbol{x_{T-1}}$ $\boldsymbol{x_T}$

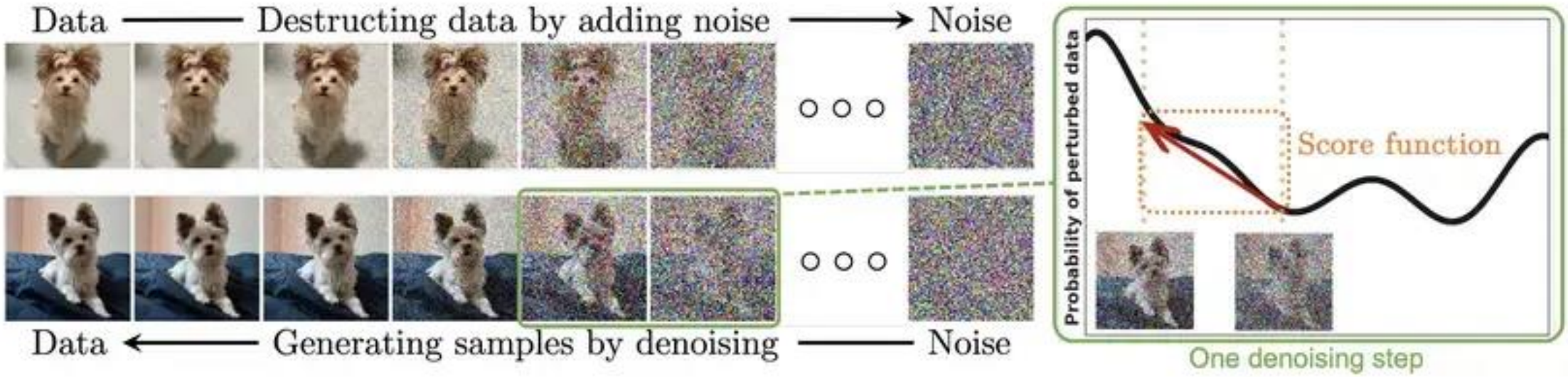
  - a **learnable** reverse diffusion process!!
  - Training diffusion model = Learning to denoise



Diffusion Models

3.. Image generation, wherein the trained model samples a random noise distribution and transforms it into a high-quality output by using the reverse diffusion process it has learned to denoise a random sample of gaussian noise.

Sample from the noise distribution $x_T \sim \mathcal{N}(0, \sigma^2 I)$, reverse the diffusion process to generate data $x_0 \sim p(x)$



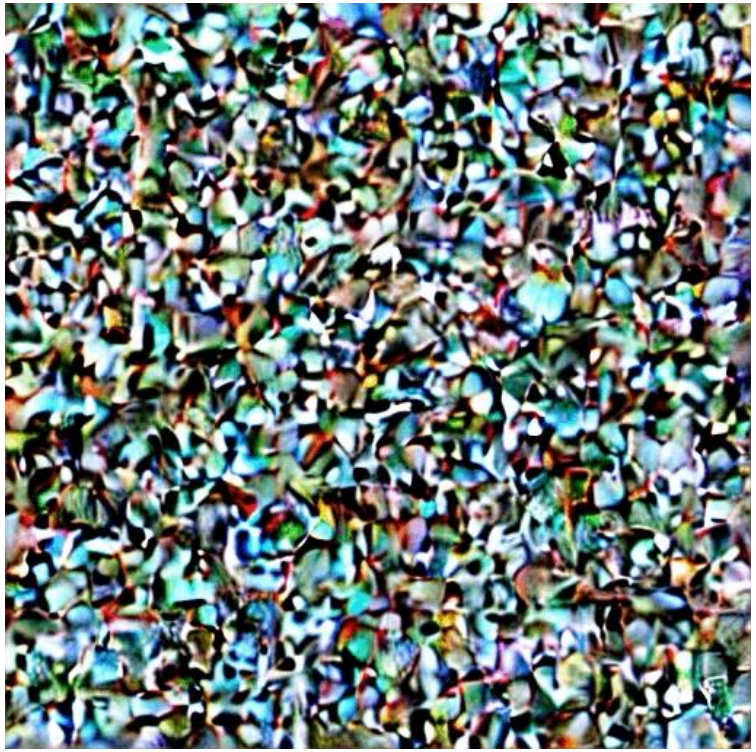
Diffusion Model

- Each step of the reverse diffusion process is carried out by a neural network, and it has the same number of steps as the forward process.

- Why use a series of small denoising steps instead of one big jump to clean the data?

  ---- Easier to learn and control the process

- The intermediate noisy images serve as latent codes and have the same size as the training images. This is one of the reasons why diffusion models can generate high-fidelity samples.



Example of Stable Diffusion in Action

Summarization

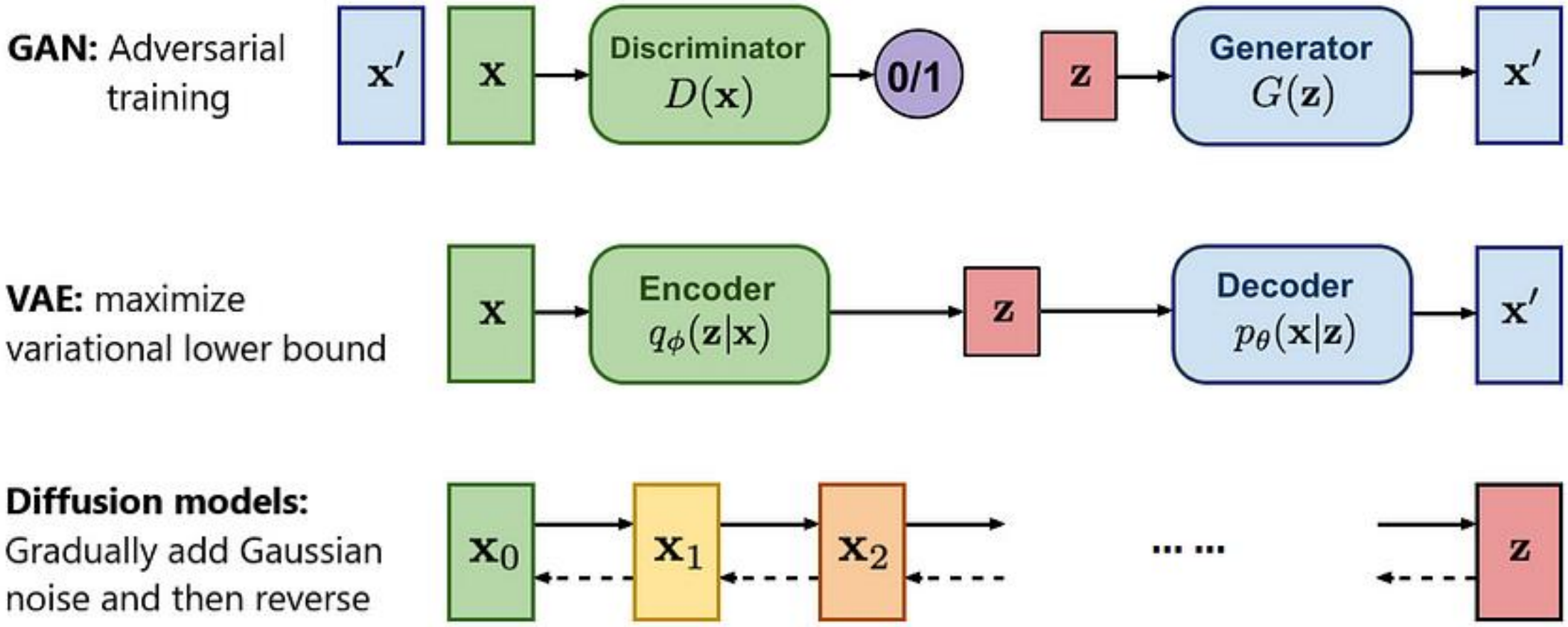
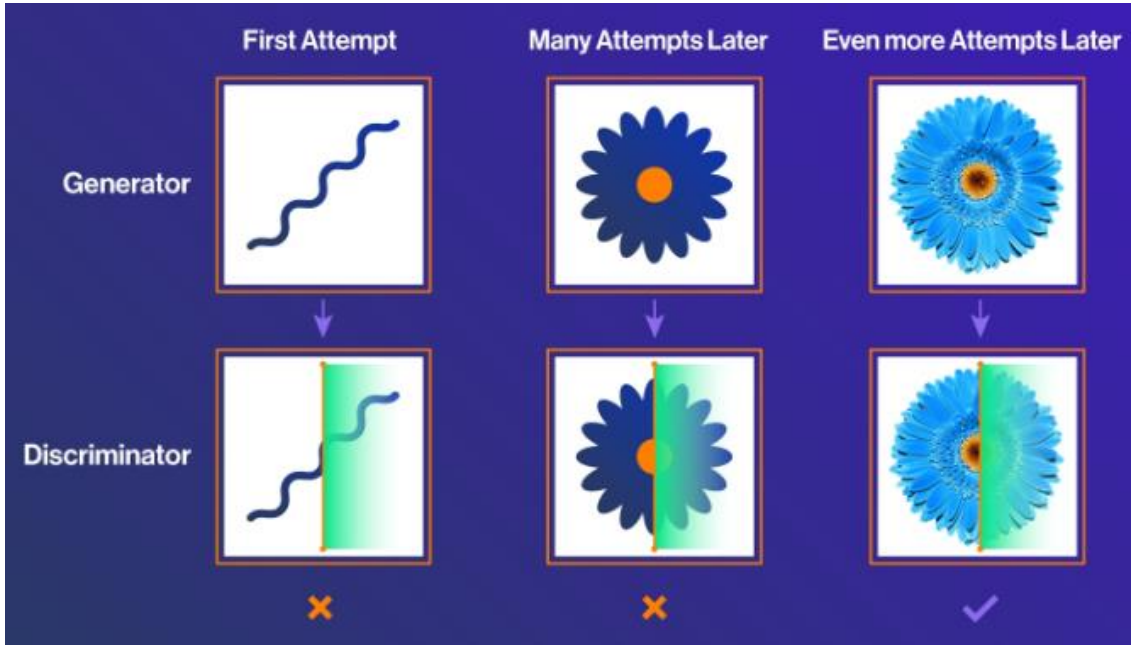
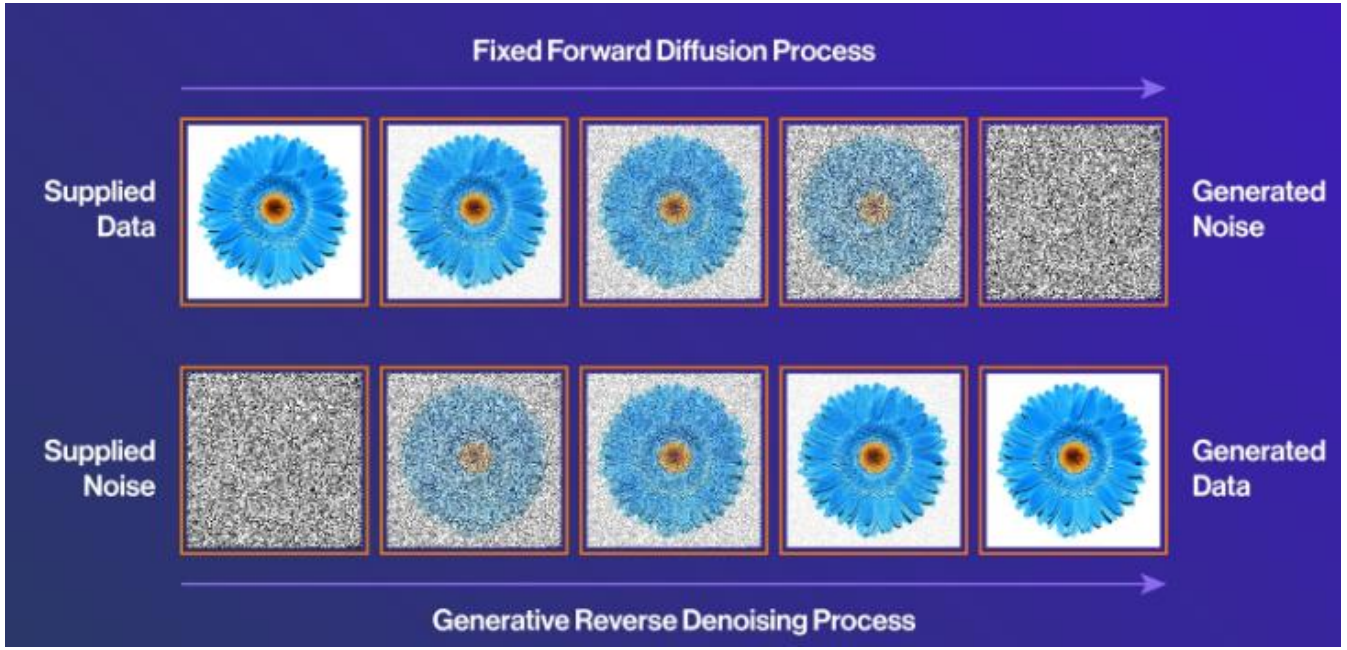


Fig from https://towardsai.net/p/machine-learning/diffusion-models-vs-gans-vs-vaes-comparison-of-deep-generative-models

GAN vs Diffusion





GAN:

- One shot generation. Fast
- Adversarial min-max objective. Hard to train. Can collapse.

Diffusion:

- Multi-iteration generation. Slow.
- Easier to control during generation.
- Simple objective, no adversary in training

https://www.sabrepc.com/blog/Deep-Learning-and-AI/gans-vs-diffusion-models

References

- Online resources:
  - Tutorial of VAE on Pytorch https://jyopari.github.io/VAE.html
  - https://text-machine-lab.github.io/blog/2020/generative-models/
  - https://www.microsoft.com/en-us/research/blog/a-deep-generative-model-trifecta-three-advances-that-work-towards-harnessing-large-scale-power/
  - https://encord.com/blog/diffusion-models/
  - Diffusion model: What are Diffusion Models? | Lil'Log
  - https://www.youtube.com/watch?app=desktop&v=fbLgFrlTnGU&ab_channel=AriSeff
  - Stable diffusion
    - Annotated & simplified code: U-Net for Stable Diffusion (labml.ai)
    - Illustrations: The Illustrated Stable Diffusion – Jay Alammar
- Survey papers:
  - Diffusion Models: A Comprehensive Survey of Methods and Applications: https://arxiv.org/pdf/2209.00796